

MODULE 2

Programming frameworks:
Introduction to Fog Computing

FOG COMPUTING:
PRINCIPLES,ARCHITECTURES AND APPLICATIONS

INTRODUCTION

- IoT environments consist of loosely connected devices.
- These devices are connected through different types of networks (heterogeneous networks).
- The purpose of an IoT environment is to collect and process data from IoT devices.
- This data is used to:
 - Mine and detect patterns
 - Perform predictive analysis or optimization
 - Make smarter decisions in a timely manner

- Data in an IoT environment is classified into two categories:
 - **Little Data (Big Stream):** Temporary (transient) data that is continuously collected from IoT smart devices.
 - **Big Data:** Permanent (persistent) data and knowledge that is stored in centralized cloud storage.
- Both Big Stream and Big Data are needed for:
 - Effective real-time analytics
 - Better decision making

- Data is collected from IoT networks consisting of smart devices.
- The collected data is sent to cloud servers for storage and processing.
- Cloud computing provides highly scalable computing platforms.
- It can be configured on demand to meet changing application requirements.
- It follows a pay-per-use model, reducing the investment needed to build analytics applications.
- However, when data is distributed across multiple locations and low latency is required, cloud-based data processing cannot meet the requirements.

Motivating Scenario

Need for an alternative computing paradigm:

- Computation should be moved to devices that are closer to the sensors than the cloud.
- These devices should have better computing capability and Internet connectivity.

Reasons:

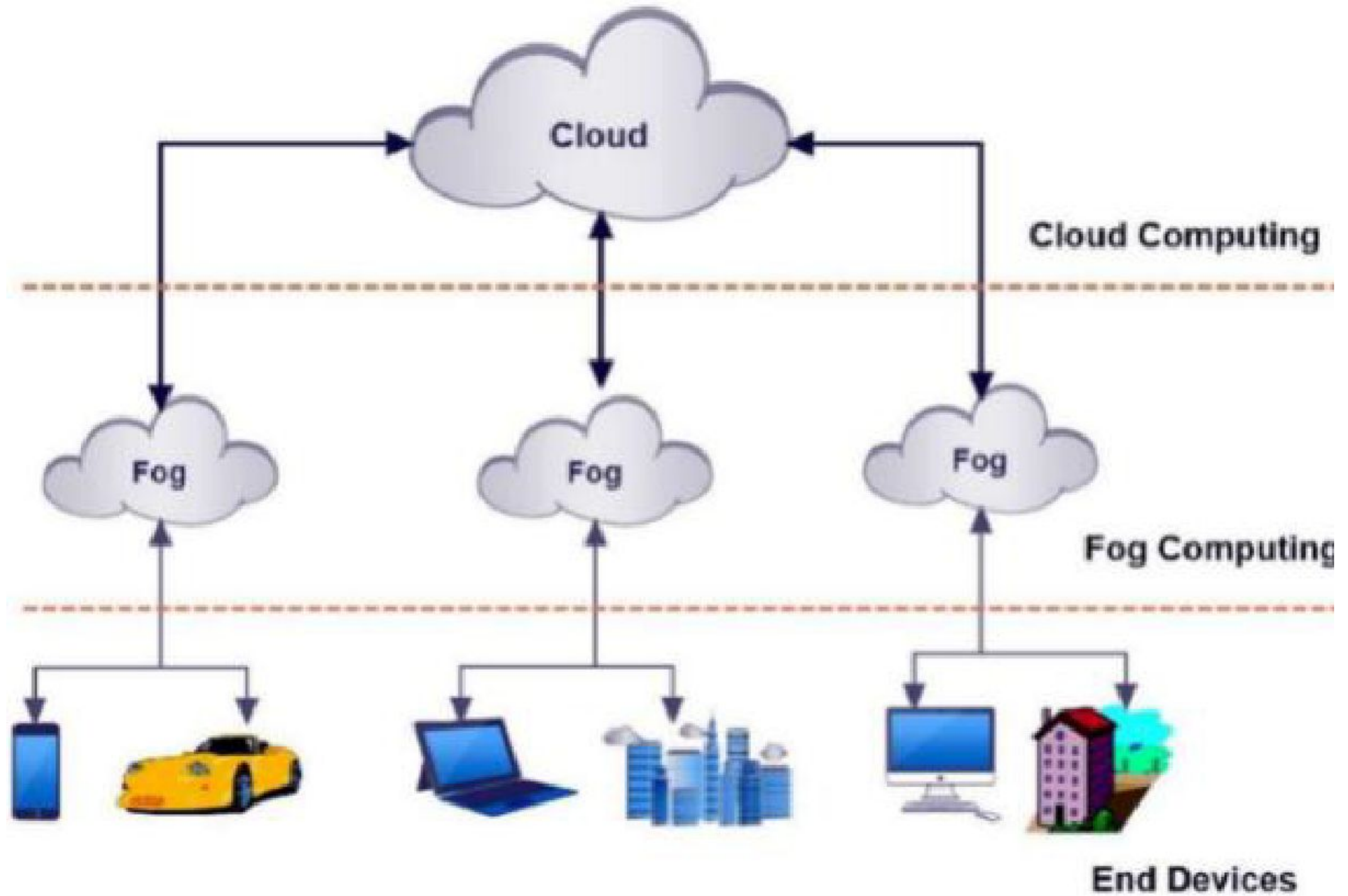
- Centralized cloud servers cannot process high-speed data flows in real time.
- Some users do not want to send and store their activity data in the cloud due to privacy concerns, even if they need statistical reports on their activities.

Cloud limitations

- Latency
- Limited Bandwidth
- Data Protection Mechanism
- Internet Connectivity

- There is a need for computationally capable devices that are closer to the sensors than the cloud.
- These devices have Internet connectivity.
- They are called edge devices because they are located at the edge of the network.
- Edge devices can:
 - Build local views of data flows
 - Aggregate data before sending it to the cloud for offline analysis

This led to the emergence of Fog Computing.



Fog Computing

- Fog computing (also called fog networking or fogging) is an architecture that uses edge devices to perform computation, storage, and communication locally before sending data over the Internet.
- It is a decentralized computing structure located between the cloud and IoT devices.
- It extends cloud computing to the network edge.
- It is ideal for IoT applications and other applications that require real-time interactions.
- Fog computing is a distributed computing paradigm that extends cloud services to the edge of the network.
- It helps in the management and programming of computing, networking, and storage services between data centers and end devices.

- In fog computing, application components run in both the cloud and edge devices.
- Edge devices can include:
 - Smart gateways
 - Routers
 - Dedicated fog devices
- These edge devices are located between the sensors and the cloud.

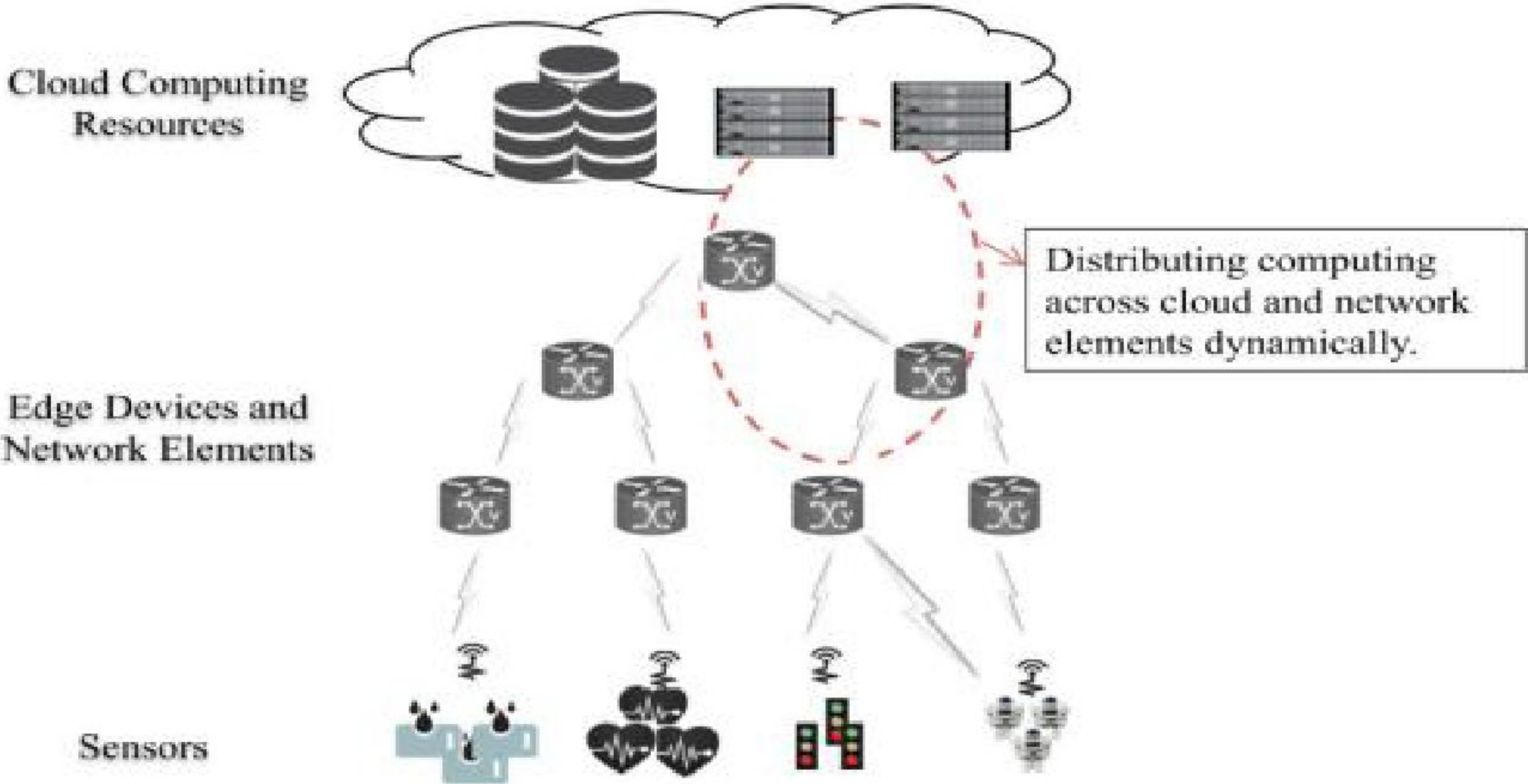
Fog computing supports :

- mobility
- computing resources
- communication protocols
- interface heterogeneity
- cloud integration
- distributed data analytics

These features help meet the requirements of applications that need:

- Low latency
- Wide and dense geographical distribution

Fog Computing is a Distributed Computing Paradigm That Extends the Cloud Services to the Edge of the Network



Advantages of Fog Computing

1. Suitable for IoT Tasks and Queries

- With the increasing number of smart devices, many requests are related to the surroundings of the device.
- Such requests can be processed without using global information from the cloud.
- Due to the local nature of some application requests, processing in fog computing is more suitable than using cloud infrastructure.
- Example: A smart-connected vehicle needs to capture events only within a short distance (about a hundred meters) around it.
- Fog computing reduces communication distance by bringing data processing closer to the edge of the network.

2. Low-Latency Requirement

- Mission-critical applications require real-time data processing.
- Example: Cloud robotics.
- Running the control system on the cloud may make the sense-process-actuate loop slow or unavailable due to communication failures.
- Fog computing performs the required processing close to the device.
- This enables faster response and real-time decision making.

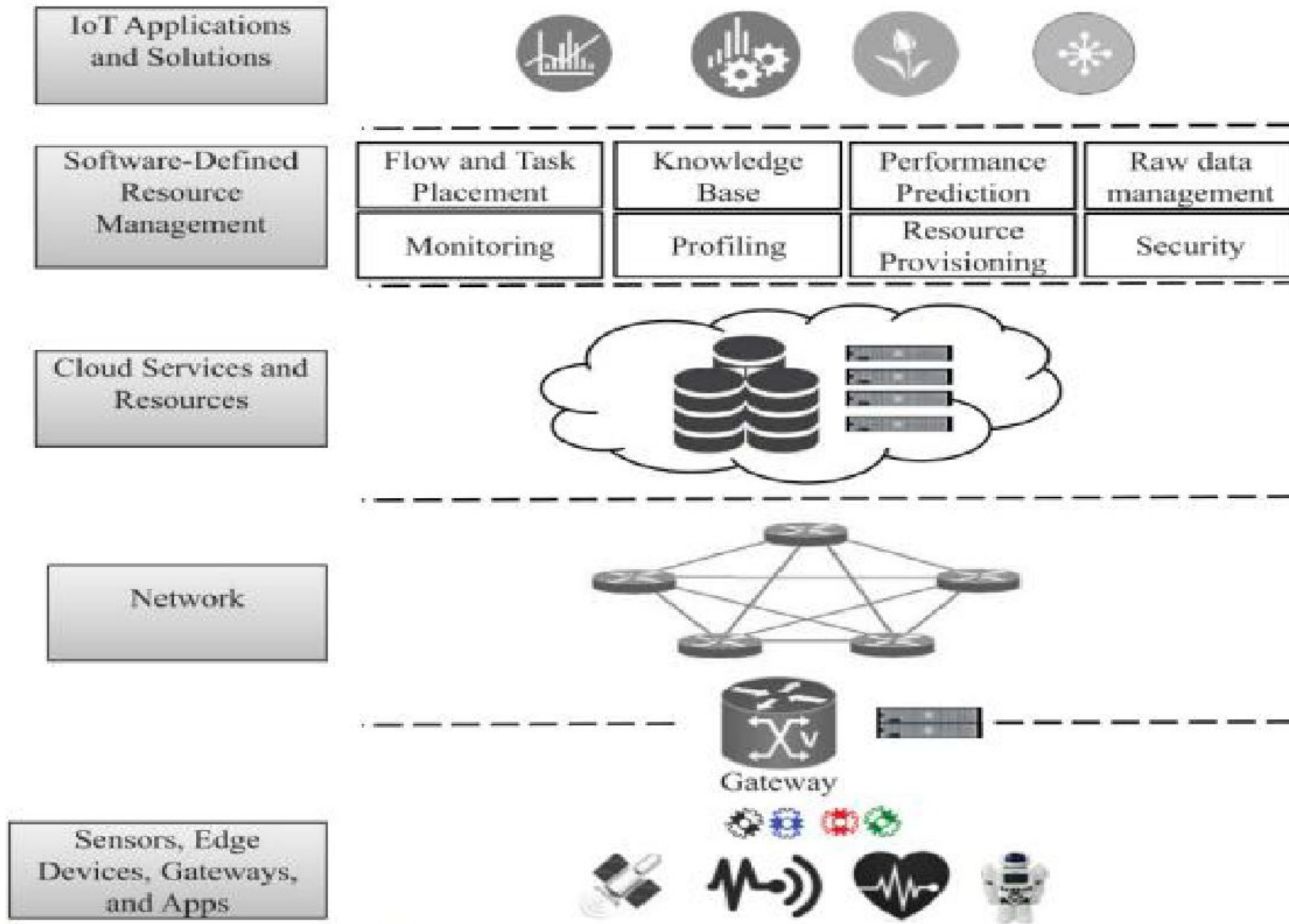
3. Reduction of Network Traffic

- Sending all the raw data to the cloud is not efficient because billions of mobile devices generate data.
- Fog computing filters and analyzes data close to the network edge.
- It also generates local data views.
- This greatly reduces the amount of data traffic sent to the cloud.

4. Scalability

- Sending all the raw data from end devices to the cloud can create a bottleneck.
- Fog computing processes data closer to the data source.
- This reduces the processing load on the cloud.
- As a result, it improves scalability by handling the increasing number of end devices (endpoints).

Reference Architecture Of Fog Computing



Fog Computing Reference Architecture

1. Bottommost layer :

End devices (sensors), Edge devices and Gateways –
it also includes apps that can be installed in the end devices to enhance their functionality.

2. Network Layer : Provides communication:

- Between devices
- Between devices and the cloud

3. Cloud Services and Resources Layer

- Supports resource management.
- Processes IoT tasks that are sent to the cloud.

4. Resource Management Software Layer

- Manages the entire fog computing infrastructure.
- Provides Quality of Service (QoS) for fog computing applications.

5. Topmost Layer

- Contains applications that use fog computing.
- Delivers innovative and intelligent services to end users.

Software-Defined Resource Management Layer

- Implements middleware-like services to optimize the use of cloud and fog resources.
- Helps applications use cloud and fog resources efficiently.
- Its main goals are to:
 - Reduce the cost of using cloud resources.
 - Maintain acceptable latency for applications.
 - Push task execution to fog nodes instead of the cloud whenever possible.

Services of the Software-Defined Resource Management Layer

1. Flow and Task Placement

- Keeps track of the available cloud, fog, and network resources.
- Identifies the best resources to execute incoming tasks and data flows.
- Communicates with the Resource Provisioning Service.

2. Knowledge Base

- Stores historical information about:
 - Application demands
 - Resource demands
- This information helps other services make better decisions.

3. Performance Prediction

- Uses information from the Knowledge Base.
- Estimates the performance of available cloud resources.
- Helps the Resource Provisioning Service decide how many resources are needed.
- This is useful when:
 - There are many tasks and data flows, or
 - Application performance is not satisfactory.

4. Raw-Data Management

- Has direct access to data sources.
- Provides data views for other services.
- Data can be accessed using:
 - Simple queries (e.g., SQL or NoSQL REST API)
 - Complex processing (e.g., MapReduce)
- The method used to generate the data view is hidden from other services.

5. Monitoring

- Monitors the performance and status of applications and services.
- Provides this information to other services whenever needed.

6. Resource Provisioning

- Responsible for allocating cloud, fog, and network resources for applications.
- Resource allocation is dynamic, based on application needs.
- The number of resources is decided using:
 - Information from other services
 - User latency requirements
 - Security credentials managed by the Security Service

7. Profiling

- Builds resource profiles and application profiles.
- Uses information from the:
 - Knowledge Base
 - Monitoring Service

8. Security

- Provides **authentication** (verifies user identity).
- Provides **authorization** (controls access to resources).
- Provides **cryptography** (protects data).
- Ensures **secure communication** for services and applications.

Applications of Fog Computing

There are variety of applications benefiting from the Fog-computing paradigm.

Major applications are,

- Healthcare
- Augmented Reality
- Caching and Preprocessing

1. Healthcare

- **Monitor Fall for Stroke Patients**

- Uses fall-detection algorithms to monitor stroke patients.
- Algorithms include:
 - Acceleration measurement
 - Time-series analysis
 - Filtering techniques
- A real-time fall-detection system divides the task between:
 - Edge devices (Fog)
 - Cloud
- This enables fast detection and quick response.

- **three-tier architecture for smart healthcare and elderly care.**
 - The architecture consists of:
 - Role model
 - Layered cloud architecture
 - Fog computing layer
 - The fog layer improves the system by providing:
 - Low latency
 - Mobility support
 - Location awareness
 - Security measures

2. Augmented Reality

- **Augmented reality (AR) applications** require very low latency.
- Even small delays can reduce the user experience.
- Therefore, fog computing is well suited for AR applications.
- In an **Augmented Brain Computer Interaction Game**:
 - Fog servers perform continuous real-time brain-state classification.
 - Cloud servers regularly update the classification models.
 - The models are updated using EEG readings collected by sensors.

- A **Wearable Cognitive Assistance System** uses Google Glass devices to help people with reduced mental ability.
- The application has compute-intensive tasks that need to be processed by an external server.
- Sending these tasks to the cloud causes high latency.
- Therefore, the tasks are offloaded to nearby fog devices for faster processing.
- The cloud is used only for delay-tolerant tasks, such as:
 - Error reporting
 - Logging

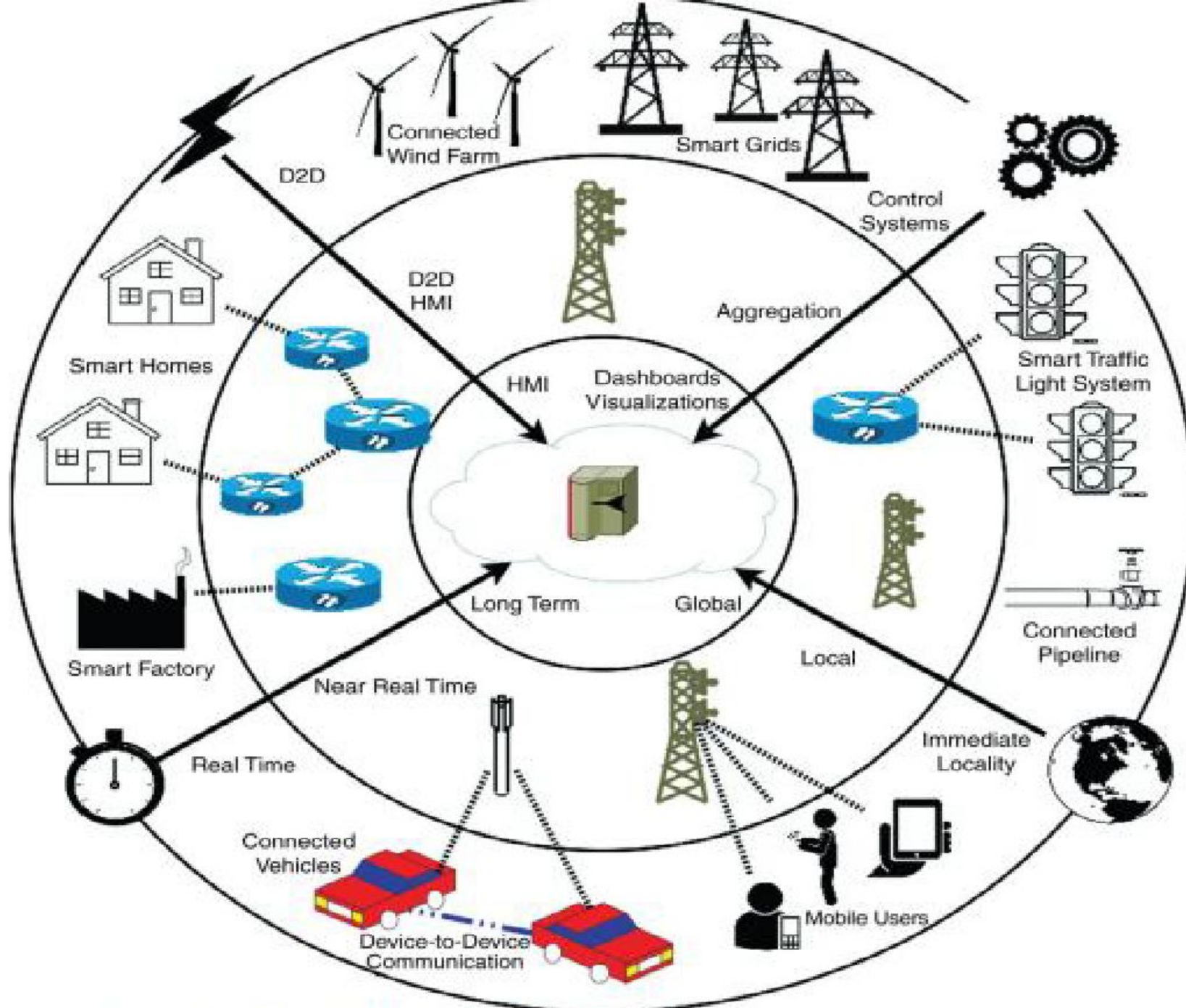
3. Caching

- The fog device performs optimizations to reduce webpage loading time.
- This helps users get faster access to the requested webpage.
- During network congestion, the edge device may provide low-resolution graphics.
- This ensures acceptable response time and improves the user experience.

4. Preprocessing

- Fog computing helps integrate IoT with cloud computing.
- This integration requires data trimming (preprocessing) before sending data to the cloud.
- Preprocessing is necessary because IoT environments generate a huge amount of data.
- Sending all the raw data to the cloud can cause:
 - Core network congestion
 - Data center congestion
- A smart gateway-based communication can be used to preprocess data and integrate IoT with cloud computing.

- Data generated by IoT devices is sent to the smart gateway:
 - Directly (one-hop), or
 - Through sink nodes (multi-hop).
- The smart gateway performs the required data preprocessing before sending the data to the cloud.
- The smart gateway is supported by fog computing services.
- Fog computing processes IoT data in a latency-sensitive and context-aware manner.
- This approach provides a richer and better user experience for IoT applications.



Range of Applications Benefiting From Fog Computing

TinyOS

Sensor Networks

- Sensor networks consist of compact, low-power wireless sensors.
- These sensors form distributed embedded systems.
- They are deployed in dynamic, constantly changing, and adaptive environments.
- Networked sensors can sense:
 - Light
 - Heat
 - Position
 - Movement
 - Chemical presence
- They collect data from the real environment and send it to traditional computers.

Embedded sensors are:

- Agile
- Self-organizing
- Resource-constrained
- Communication-centric

They are used in many applications, including:

- Context-aware monitoring
- Life science experiment monitoring
- Disaster management
- Other monitoring applications

- Sensor networks consist of thousands of tiny, low-power sensor nodes.
- Each node runs concurrent and reactive programs.
- These nodes have limited memory and power.

Challenges in Sensor Networks:

- Limited resources
- Event-centric concurrent applications
- Low-power operation

These challenges led to the design of TinyOS.

TinyOS

- TinyOS was developed by four Computer Science graduate students at the University of California, Berkeley.
- Its development was supported by:
 - Defense Advanced Research Projects Agency (DARPA)
 - National Science Foundation (NSF)
 - Intel Corporation

History of TinyOS

- 1999: First TinyOS platform and OS implementations are developed at Berkeley.
- 2000: Berkeley designs TinyOS version 0.43 is made available to the public via SourceForge.
- 2001: Berkeley develops the mica platform and releases TinyOS version 0.6.
- February 2002: Berkeley distributes 1000 mica nodes to other participants in the NEST project.
- April 2002: Work on the nesC programming language begins as a collaboration between Intel Research and UC Berkeley.
- September 2002: TinyOS version 1.0, implemented in nesC, is released.
- August 2003: TinyOS version 1.1 is released, which includes new nesC features including data race detection.

- September 2003 – December 2005: TinyOS begins a periodic minor release process.
- June 2004: Working group forms on next steps for TinyOS, based on experiences porting to new platforms. Group agrees to start work on 2.0.
- July 2005: NEST project concludes.
- December 2005: TinyOS 1.1.15, the last 1.1 version, is released.
- November 2006: TinyOS 2.0 released at the SenSys conference in Boulder, CO.
- August 2008: TinyOS 2.1.0 released.
- April 2010: TinyOS 2.1.1 released.
- August 2012: TinyOS 2.1.2 released

Introduction

- TinyOS is a flexible, application-specific operating system for sensor networks.
- It is specially designed for real-time embedded sensors.
- It is a very small operating system.
- It focuses on saving power.
- TinyOS software is implemented using the NesC programming language, which has a syntax similar to C.
- TinyOS is an event-driven operating system useful for behavior-based applications.
- The main aim is to separate power management, networking, and sensor measurements from the core system.

Requirements Motivating the Design of TinyOS

- TinyOS supports complex and concurrent programs with very low memory requirements.
- Many applications can fit within 16 KB of memory.
- The core operating system is only about 400 bytes.
- It provides efficient and low-power operation.

Four Main Requirements

- Limited Resources
- Reactive Concurrency
- Flexibility
- Low Power

Requirements Motivating the Design of TinyOS

1. Limited Resources

- Motes have very limited physical resources because they are designed to be:
 - Small in size
 - Low cost
 - Low power consuming
- Current motes have approximately:
 - 1-MIPS processor
 - Tens of kilobytes of storage
- Moore's Law is expected to help reduce the size and cost of motes rather than increase their capabilities.
- Current motes are measured in square centimeters.

2) Reactive Concurrency

- In a typical sensor network application, a node performs many tasks:
 - Senses the environment using sensors.
 - Controls devices using actuators.
 - Processes data locally.
 - Transmits data.
 - Routes data for other nodes.
 - Participates in distributed processing tasks, such as:
 - Statistical aggregation
 - Feature recognition
- Many activities, such as radio management, require real-time responses.
- Therefore, TinyOS needs a concurrency management approach that:
 - Reduces possible programming errors
 - Respects resource limitations
 - Meets timing requirements

3) Flexibility

- Different hardware and applications require a flexible operating system.
- The fast rate of innovation also requires the OS to be flexible and adaptable.
- The OS should be:
 - Application-specific to reduce memory usage and power consumption.
 - Independent of the boundary between hardware and software.
- The OS should support:
 - Fine-grain modularity
 - Interposition
- This makes reuse and innovation easier.

4) Low Power

- Power consumption is a continuous challenge in sensor networks.
- Low-power operation is a key goal in mote design.
- Energy harvesting provides promising solutions, but tiny motes can harvest only microwatts of power.
- This is not enough for continuous operation, even for energy-efficient designs.
- Therefore, TinyOS must:
 - Support extremely low-power operation.
 - Provide flexible power management for different sensor network applications.

TinyOS

- TinyOS uses a component-based programming model.
- This model is implemented using the NesC language.
- TinyOS is not a traditional operating system.
- It is a programming framework for embedded systems.
- It provides a set of components that help build an application-specific operating system for each application.
- A typical TinyOS application is about 15 KB in size.
- The base OS is only about 400 bytes.
- A large application, such as a database-like query system, can be about 64 KB in size.

The current version of TinyOS provides many components for application developers, including:

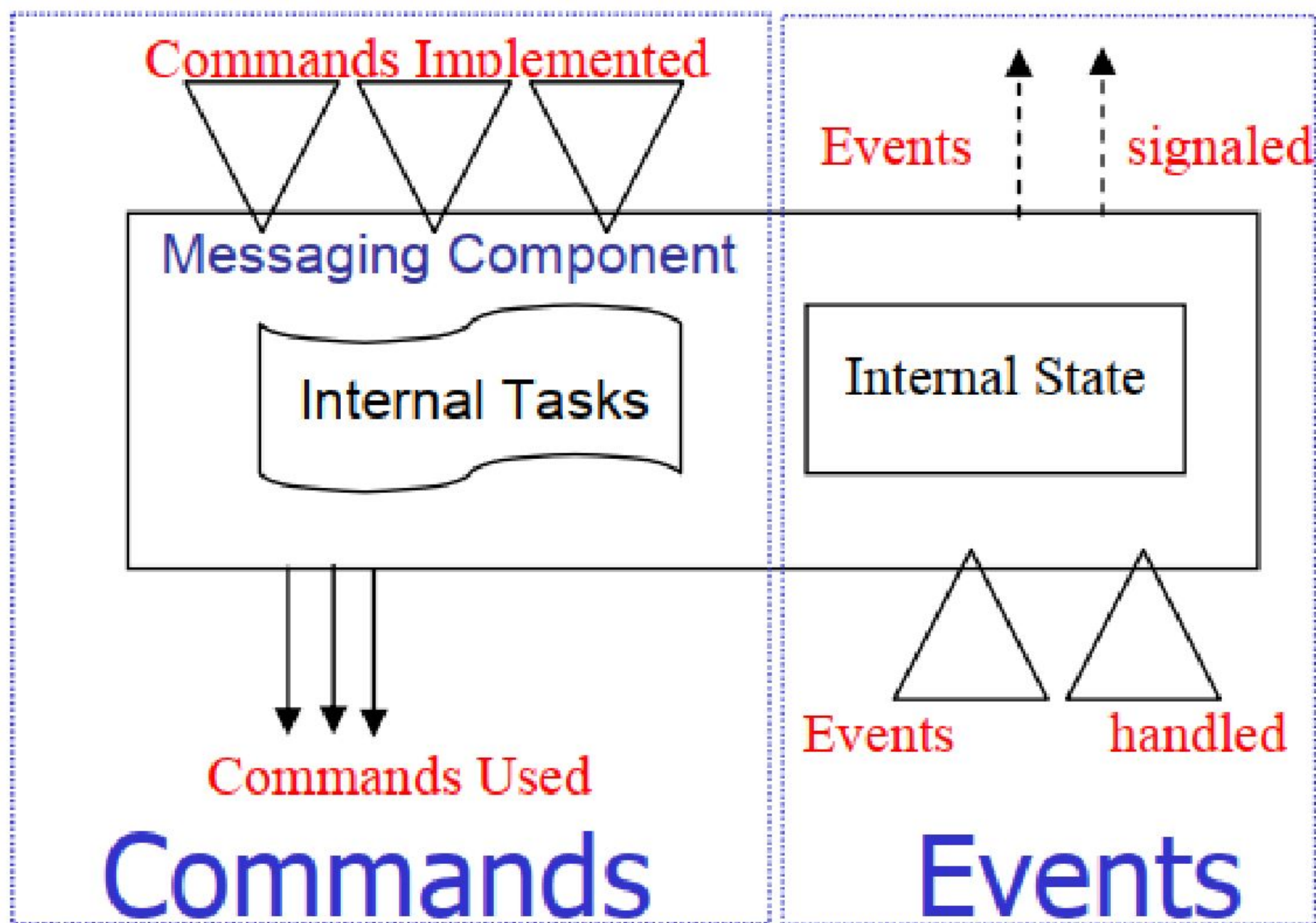
- Sensor abstractions
- Single-hop networking
- Ad-hoc routing
- Power management
- Timers
- Non-volatile storage

Component Model

- TinyOS uses a component-based programming model, provided by the NesC language.
- Components encapsulate a specific set of services.
- These services are specified using interfaces.
- TinyOS consists of:
 - A set of reusable system components
 - A task scheduler
- An application connects different components using a wiring specification.
- The wiring specification is independent of component implementation.
- It defines the complete set of components used by the application.

- A TinyOS program is a graph of components.
- Each component is an independent computational unit that provides one or more interfaces.
- Components have three computational abstractions:
 - Commands
 - Events
 - Tasks
- Commands and events are used for communication between components.
- Tasks are used to handle concurrent activities within a component.

- Command: A request to a component to perform a service, such as starting a sensor reading.
- Event: Indicates that the requested service has been completed.
- Task: Represents internal concurrency within a component and can access only the state of that component.
- The standard TinyOS task scheduler uses:
 - Non-preemptive scheduling
 - FIFO (First-In, First-Out) scheduling



Structure of a Component in TinyOS

- A component has two types of interfaces:
 - Interfaces it provides
 - Interfaces it uses
- These interfaces define how a component interacts with other components.
- An interface generally represents a service, such as sending a message.
- Interfaces are bidirectional and contain both:
 - Commands
 - Events
- Command: A function implemented by the provider of an interface.
- Event: A function implemented by the user of an interface.

TinyOS Computational Concepts

1. Events

- Time critical
- Caused by interrupts(timer, ADC, Sensors)
- Short duration

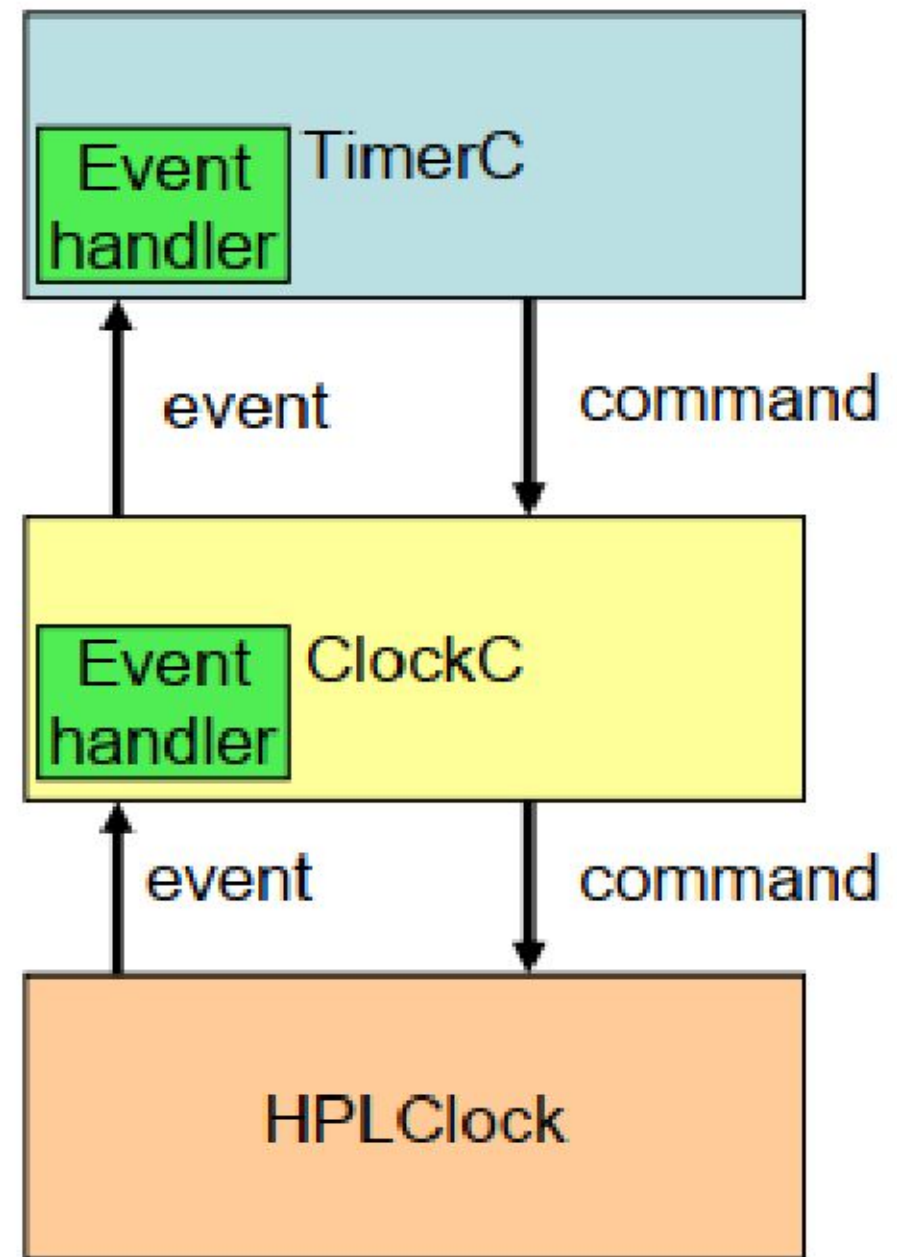
2. Commands

- Request to a component to perform service(eg. Start sensor reading)
 - **Example:** Start sensor reading.
- Non-Blocking, need to return status.
- Time-consuming work is postponed by posting a task.
- Can call lower level commands

3. Tasks

- Time-flexible: Tasks are used for delayed or non-urgent processing.
- Tasks run sequentially according to the TinyOS scheduler.
- A task runs to completion before another task starts.
- Tasks can be interrupted by events.

- Components are wired together by connecting users with providers
 - Forms a hierarchy
- Commands:
 - Flow downwards
 - Control returns to caller
- Events:
 - Flow upwards
 - Control returns to signaler
- Events can call Commands but not vice versa



```
{  
...  
status = call interfaceName.CmdName(args)  
...  
}
```

```
event EvtName(args) {  
...  
return status;  
}
```

Interface user

```
command CmdName(args) {  
...  
return status;  
}
```

```
{  
...  
status = signal interfaceName.EvtName(args)  
...  
}
```

Interface provider

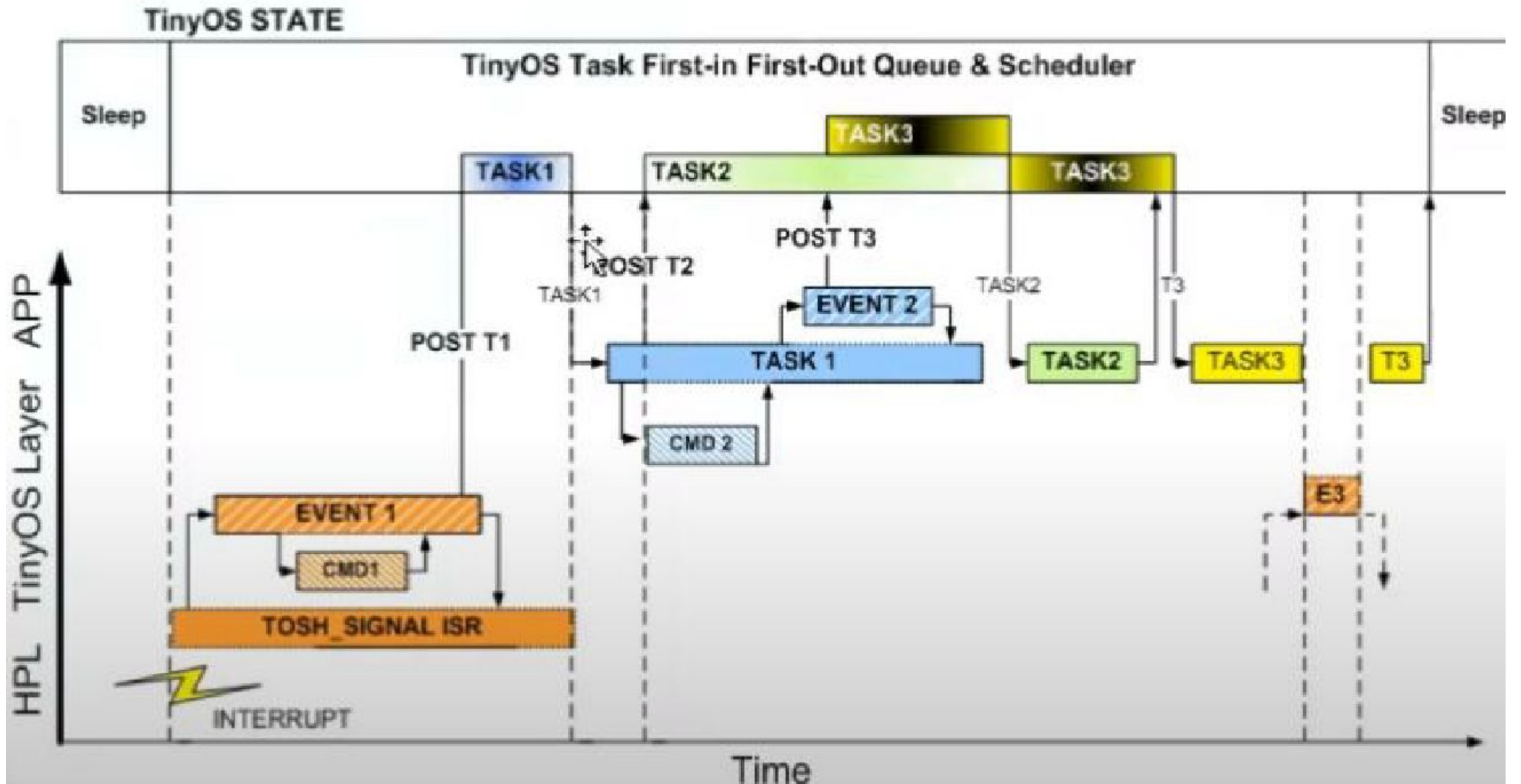
- Commands “**call down**” towards the hardware components
- Event “**call up**” towards application components

Execution Model and Concurrency

- Sensor networks are event-centric, so events can arrive at any time.
- These events must work properly with the ongoing computation.
- There are **two main approaches** to handle such events:
 - 1. Enqueue the work**
 - When an event arrives, its work is placed in a queue.
 - The work is executed later.
 - 2. Execute immediately**
 - The event handler is executed immediately when the event arrives.
 - This is similar to active messages.
 - This approach is useful because some events are time-critical.

- The execution model consists of:
 - Run-to-completion tasks – perform the ongoing computation.
 - Interrupt handlers – are triggered asynchronously by hardware.
- A program submits a task to the scheduler using the post operator.
- The scheduler executes tasks according to its scheduling policy.
- Once a task starts, it must run to completion before another task runs.
- The standard TinyOS scheduler follows a FIFO (First-In, First-Out) policy.

TinyOS Execution Model



Concurrency

TinyOS has **two threads of execution**:

1. Tasks

- Used for deferred execution.
- Tasks cannot pre-empt other tasks. A task runs until it is completed.

•Hardware Event Handler

- Responds to hardware interrupts.
- Interrupts can pre-empt tasks.

2. Scheduler

- TinyOS uses two levels of scheduling:
 - Interrupts (vector)
 - Tasks (queue)
- The task queue follows FIFO (First-In, First-Out).
- When there is no event or command running and the task queue is empty, the scheduler puts the processor to sleep to save power.

TinyOS Theory of Execution: Events and Tasks

Consequences of an event

- Runs to completion
- Pre-empt tasks

What can an event do?

- Signal events
- Call commands
- Post tasks

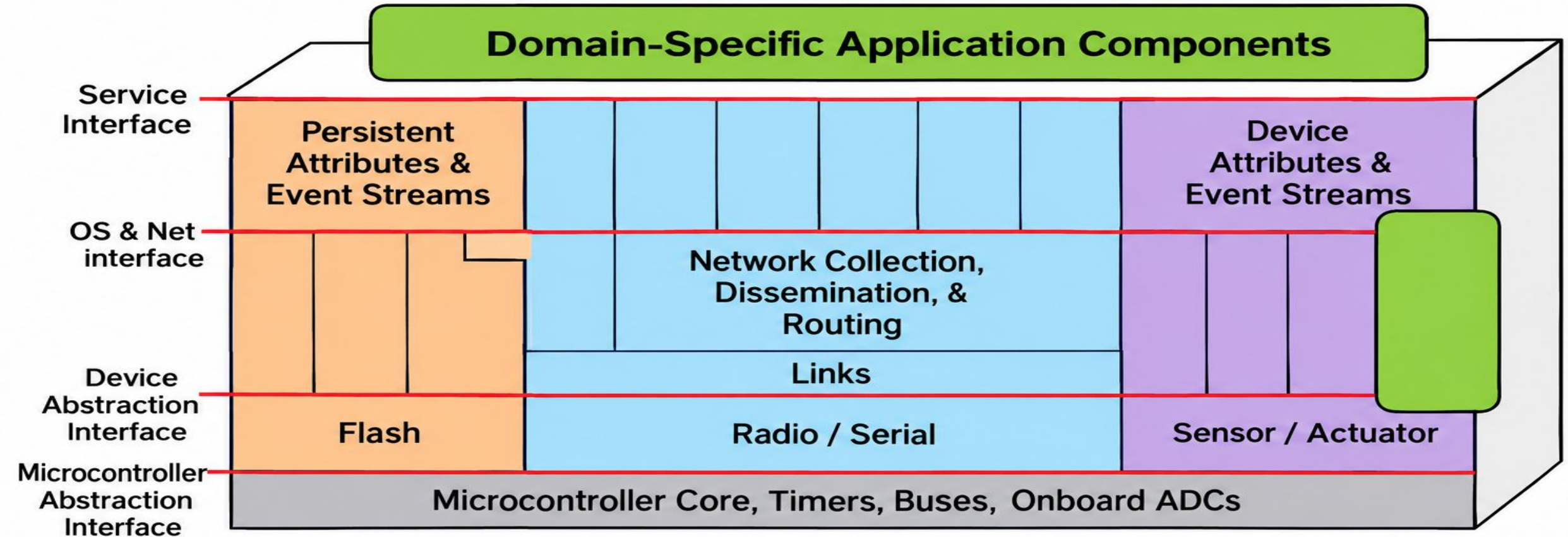
What can initiate (post) tasks?

- Command, event or another task

Consequences of a Task

- There is no pre-emption mechanism between tasks.
- Keep the code in small execution pieces so that other tasks are not blocked for too long.
- For long operations, create separate tasks for each operation instead of using one large task.

TinyOS Architecture



Hardware



TelosB



MicaZ



Intel Mote2

- TinyOS follows the monolithic architecture.
- It uses a component model.
- Different components are connected with the scheduler according to the application requirements.
- These components form a static image that runs on the mote.
- A component is an independent computational entity that provides one or more interfaces.
- Components have three computational abstractions:
 - Commands
 - Events
 - Tasks

- Commands and events are used for communication between components.
- Tasks are used to express concurrency within a component.
- A command is a request to perform a service.
- An event signals the completion of the service.
- TinyOS uses a single shared stack.
- There is no separation between kernel space and user space.